



Rust Programming

Skylar Coelho

February 4, 2026

1 Rust Basics

1.1 Installing Rust

Install Rust from the [Rust website](#). Once you have rust installed, check that the Rust compiler is set up correctly by running

```
1 $ rustc --version
```

Rust can be updated using rustup, the included tool chain for installing and maintaining Rust. The version of rustup can be checked in a similar matter. Rust can be updated via rustup by running

```
1 $ rustup --version
2 $ rustup update
```

1.2 Cargo Basics

Rust runs typically from a main function created by initializing the project using cargo. Initialize a project using cargo

```
1 $ cargo new project_name
```

If the directory already exists, you can build a project in that directory using

```
1 $ cargo init
```

You can add dependencies, like external libraries from [crates.io](#), by running

```
1 $ cargo add package_name
```

Finally, when you're ready, you can build your project for debugging or release using cargo as well via

```
1 $ cargo build # compiles for debug by default
2 $ cargo build --release # for building using the release profile
3 $ cargo run # compiles for debug and runs the executable
4 $ cargo run --release # compiles and runs the release profile executable
```

Additionally, you can always compile your Rust script using the Rust compiler itself and then running the produced executable

```
1 $ rustc main.rs # compile
2 $ ./program_name # run the produced executable
```

1.3 Basic Setup

Typical Rust programs will contain the following code snippet within `main.rs` by default

```
1      fn main(){
2          println!("Hello, world!")
3      }
```

The code contained in the `main` function is what gets compiled and called by the typical Rust compilation actions. Note that the default function setup contains the macro `println!` which prints the string literal `"Hello, world!"` to the console. We know this is a macro because the standard naming scheme for macros in Rust is to leave an `!` at the end of a function name.

1.4 Including Libraries

Certain libraries are packaged with Rust, known as the *standard library*, shortened to *std*. These packages do not need to be included as dependencies via cargo and can be brought into your code via the following syntax

```
1      use std::io
```

The double colon `::` is similar to the dot `.` notation from other languages where the above brings in the `io` library from the standard library.